SUBSCRIPTION INTELLIGENCE

& REVENUE MANAGEMENT PLATFORM

Detailed Project Proposal & Product Documentation

Hackathon: Subscription Management System
Prepared from the Brightloop Software challenge brief and five public GitHub references
Research date: 31 October 2026

PRODUCT VISION • COMPETITIVE REVIEW • USPs • WEBSITE STRUCTURE
WORKFLOWS • ARCHITECTURE • FEASIBILITY • ROADMAP

Document Guide

1. Executive Summary
2. Problem Statement and Background
3. Proposed Product and Vision
4. Research Method and Reference Review
5. Comparative Feature Matrix
6. Gap Analysis and Product Response
7. Proposed USPs and Differentiation
8. Functional Requirements and Website Structure
9. End-to-End Workflows and Business Rules
10. Analytics, Metrics and Intelligence
11. Technical Architecture
12. Security, Privacy and Compliance Considerations
13. Feasibility and Implementation Roadmap
14. Demonstration Script for Judges
15. Success Metrics and Validation
16. Future Scope
17. Conclusion
18. References

1. Executive Summary

This proposal defines a unified Subscription Intelligence and Revenue Management Platform for subscription-based businesses. It is designed around the complete business lifecycle: product and plan configuration, customer acquisition, quotations, subscription activation, recurring billing, tax and discount calculation, payment recording, reconciliation, renewal automation, customer self-service and revenue analytics.

The platform responds directly to the Brightloop Software challenge. Brightloop currently relies on spreadsheets, manually remembered renewals, hand-created invoices, manual bank-statement matching and quarterly reporting. These fragmented processes cause missed renewals, billing errors, delayed collections and poor visibility into recurring revenue.

The proposed solution combines the strongest relevant ideas found in five reference repositories, while avoiding unsupported claims that those projects lack features. It differentiates through a unified workflow, transparent billing, payment reconciliation, customer self-service, flexible pricing and decision-support analytics.

2. Problem Statement and Background

Subscription businesses manage recurring customer relationships rather than one-time transactions. Their operations connect product configuration, pricing, contracts, subscription states, billing schedules, invoices, taxes, payments, renewals and financial reporting. When these activities are spread across spreadsheets and disconnected tools, errors and operational delays accumulate.

2.1 Current-state problems

Current process	Observed problem	Business impact
Subscriptions tracked in spreadsheets	No single reliable source of truth; manual updates	Inconsistent records and poor visibility
Renewals remembered by staff	No dependable renewal queue or ownership	Missed renewals and avoidable churn
Invoices typed manually	Repeated data entry and calculation risk	Incorrect invoices and delayed billing
Payments matched using bank statements	No structured allocation to invoices	Unidentified receipts and incorrect balances
Revenue reports built quarterly	Metrics assembled manually and late	Slow decisions and limited forecasting

2.2 Challenge scope from the supplied brief

The supplied Brightloop brief requires registration and password reset; role-based access for admin, staff and customers; admin-only staff creation; products with type, sales price, cost price, notes and variants; plans with daily, weekly, monthly or yearly billing; quantity and availability rules; pause, close, renew and expiry behaviour; quotation templates; subscription statuses Draft, Quotation, Confirmed, Active and Closed; automated invoice generation; tax rules; invoice states Draft, Confirmed and Paid; fixed and percentage discounts with eligibility and usage limits; payment records; overdue visibility; and finance dashboards.

The brief also sets a performance target: pages should respond in under two seconds even with thousands of subscriptions. This is a target to validate through testing, not a performance claim to make without measurements.

3. Proposed Product and Vision

Proposed product name

Unified Subscription Intelligence & Revenue Management Platform

A business operating platform that helps subscription companies create, sell, manage, bill, retain and analyze customer subscriptions through one connected system, while giving customers a self-service portal to manage their own subscriptions and billing information.

3.1 Product vision

Create one reliable source of truth for the commercial and financial lifecycle of every subscription. Each quotation, plan, subscription, invoice, payment, adjustment and renewal should be traceable to the same customer and have a clear status, owner and history.

3.2 Target users

User / role	Primary responsibilities	Access boundary
Platform / business admin	Configure products, plans, taxes, discounts, team, policies and reports	Full administrative access
Sales staff	Manage prospects/customers, prepare quotations and convert accepted offers	Sales and assigned customer records
Operations staff	Monitor subscriptions, renewals, exceptions and customer requests	Operational actions permitted by policy
Finance staff	Review invoices, payments, refunds, reconciliation and reports	Financial records and approved actions
Customer	View own subscriptions, invoices, payments, usage and permitted plan changes	Own account and own records only

3.3 Intended business segments

The platform is suitable in concept for SaaS companies, membership businesses, education and training providers, managed-service providers, digital service businesses and other organizations with recurring contracts. Industry-specific tax, compliance and payment requirements must be configured and validated before production use.

4. Research Method and Reference Review

The five supplied GitHub repositories were reviewed using their public repository pages and README-level documentation available at the time of preparation. This is a documentation-based comparison, not a complete source-code audit, deployed-product test or security review. A feature not mentioned in a README is marked 'not established in reviewed documentation', not automatically 'unsupported'.

4.1 Reference 1 — FlexPrice

Repository: https://github.com/flexprice/flexprice

Documented focus: developer-oriented monetization infrastructure for usage-based, credit-based and hybrid pricing. Its README describes real-time usage metering, credit grants and top-ups, pricing plans, feature entitlements and limits, subscription invoicing, proration, overages, API/SDK workflows, self-hosting and integration with payment, CRM, CPQ and accounting systems.

What to learn: flexible pricing models; usage events; credits; feature entitlements; clear usage-to-invoice traceability; API-first architecture. Product opportunity: present these capabilities through a simpler business workflow and customer portal alongside quotations, staff roles, finance operations and payment reconciliation. Do not claim FlexPrice lacks those capabilities unless separately verified.

4.2 Reference 2 — Huhusmang Subscription Management

Repository: https://github.com/huhusmang/Subscription-Management

Documented focus: managing and tracking recurring expenses and renewals. Its public README describes a dashboard with monthly/yearly expense statistics, upcoming reminders and category analysis; subscription add/edit/status management and batch import; and payment history with search and CRUD operations. Repository structure includes separate server and client code.

What to learn: simple recurring-payment overview, upcoming reminders, expense categorization, payment history and convenient subscription entry. Product opportunity: translate customer-side visibility into a seller-side business system with products, quotes, contracts, invoice generation, taxes, payment allocation and role-aware operations. The README-level evidence does not establish a complete business billing workflow.

4.3 Reference 3 — BillaBear

Repository: https://github.com/billabear/billabear

Documented focus: a self-hostable subscription management and billing system integrating with Stripe and exposing a REST API. Its README lists tax rules and thresholds, workflows, plans, subscriptions, invoices, hosted checkout, payment links, quotes, mass subscription changes, subscription/LTV/churn reports, email-provider integrations and templates, document templates, multiple brands/currencies/languages, add-ons, webhooks, dunning, vouchers and credit notes. It describes fixed, package, tiered, graduated, fixed-plus-unit and seat pricing.

What to learn: end-to-end billing operations, tax configurability, quotes, retryable workflows, document templates, dunning and financial reporting. Product opportunity: differentiate through an integrated and understandable user experience, explicit payment reconciliation, customer-facing subscription visibility and decision-support analytics. BillaBear already covers many advanced billing functions; do not claim those as unique.

4.4 Reference 4 — Subflow

Repository: https://github.com/ridemountainpig/subflow

Documented focus: a personal recurring-expense tracker. Its README lists centralized subscription management, monthly averages for yearly plans, calendar-based charge tracking, spending charts, localization and multi-currency support, cloud sync, email reminders,

co-subscriber management, a Raycast extension and smart subscription entry from pasted details or receipts.

What to learn: calendar-based renewal visibility, spending analytics, reminders, shared subscription context and low-friction data entry. Product opportunity: provide a similar clarity of experience to business customers, but connect it to the business's authoritative subscription, invoice and payment records.

4.5 Reference 5 — AnuragThePathak Subscription Management

Repository: https://github.com/AnuragThePathak/subscription-management

Documented focus: a Go backend for recurring subscriptions with automated billing, renewal scheduling and email notifications. Its README describes clean architecture, MongoDB, Redis, Asynq task queues, schedulers and workers, transactional subscription/bill updates, deduplication and retries, JWT access/refresh tokens, rate limiting, distributed tracing, Prometheus metrics, Loki logs, Jaeger traces, health/readiness endpoints and unit/integration testing.

What to learn: reliable background processing, transactional billing, retries, deduplication, observability, clean boundaries and testability. Product opportunity: implement these reliability principles in the chosen stack while delivering a broader business-facing product experience.

5. Comparative Feature Matrix

Legend: D = explicitly documented; P = partial, adjacent or limited to a different user context; ND = not established in the reviewed README/repository documentation. ND is not proof that a feature is absent from source code or integrations.

Capability	FlexPrice	Huhusmang	BillaBear	Subflow	Anurag
Usage metering / consumption pricing	D	ND	P	ND	ND
Credits / top-ups / entitlements	D	ND	D (vouchers/credits notes)	ND	ND
Fixed recurring plans	D	P (tracked expenses)	D	P (tracked recurring expenses)	D
Personal expense tracking	ND	D	ND	D	ND
Upcoming payment calendar / reminders	P	D	P (workflow/dunning)	D	D (scheduler/email)
Quotation / quote workflows	ND	ND	D	ND	ND
Tax rules	ND	ND	D	ND	ND
Invoice generation / invoicing	D	P (payment history)	D	ND	D (bills)
Payment collection / provider integration	D (integrations)	ND	D (Stripe etc.)	ND	ND (bill creation documented)
Payment-to-invoice reconciliation	P (billing traceability)	ND	ND (not explicit in README)	ND	ND (not explicit in README)
Discounts / vouchers	P (promotional credits)	ND	D	ND	ND
Customer-facing portal / checkout	P (dashboard/API)	P (personal dashboard)	D (hosted checkout/customer frontend)	D (personal app)	P (API users)
Revenue / churn analytics	D (usage/billing reporting)	P (expense analytics)	D	P (spending analytics)	P (technical observability)
Dunning / retry workflows	P (workflow/invoice operations)	ND	D	ND	D (retryable jobs)
Background jobs / queue reliability	D (platform architecture)	ND	D (workflows)	ND	D
Monitoring / tracing	ND	ND	ND	ND	D
Multi-currency / localization	ND	P (currency)	D	D	ND
Business roles / permissions	ND	ND	ND	P (shared users)	P (JWT owner endpoints)

Interpretation: BillaBear is the closest business billing reference; FlexPrice is strongest for flexible usage and credit monetization; Anurag's project is strongest as a backend reliability reference; Huhusmang and Subflow contribute customer/expense tracking ideas. Our proposal should combine these domains into one coherent workflow rather than claim that every feature is absent from all competitors.

6. Gap Analysis and Product Response

Observed design gap / opportunity	Why it matters	Our proposed response	Priority
Different products serve different contexts	Business billing and personal expense tracking are not the same workflow	One platform with role-specific admin, staff, finance and customer experiences	Must
Business lifecycle can span multiple tools	Product, quote, subscription, invoice and payment records must stay linked	End-to-end lifecycle with shared customer and subscription IDs	Must
Payment records do not automatically prove invoice allocation	Unmatched and partial receipts distort receivables	Payment allocation, partial-payment support and unmatched-payment queue	Must
Flexible pricing can be complex for non-technical teams	Pricing changes otherwise require engineering effort	Guided fixed, seat, tiered, usage, credit and hybrid plan builder	Should
Customer visibility is a separate experience concern	Customers need to understand charges and manage permitted changes	Self-service portal for subscriptions, usage, invoices and payments	Must
Operational failures need explicit recovery	A failed job or payment must not disappear silently	Retry queue, failure state, alerts, audit history and idempotency	Must
Metrics can be descriptive but not actionable	Teams need to know what needs attention next	Renewal queue, overdue worklist, churn-risk signals and forecast scenarios	Should
Enterprise complexity can overwhelm a small business	Too many controls reduce adoption	Role-based navigation, sensible defaults and progressive configuration	Should

The gap analysis identifies product opportunities based on the different focus areas and documented coverage of the references. It does not assert that every listed opportunity is absent from every reference implementation.

7. Proposed USPs and Differentiation

USP 1 — Unified subscription lifecycle

Connect product → plan → quotation → subscription → invoice → payment → renewal/upgrade/cancellation. Every transition has a status, timestamp, actor and activity record. This directly reduces fragmented operations.

USP 2 — Transparent billing and payment reconciliation

Show invoice total, amount paid, amount outstanding, payment references and allocation status. Support partial payments and multiple payments per invoice. Import a CSV statement for assisted matching; unmatched transactions remain in a review queue. Never mark an invoice fully paid merely because some payment was received.

USP 3 — Flexible monetization engine

Support fixed recurring, per-seat, tiered/volume, usage-based, prepaid credit and hybrid pricing. For a hackathon MVP, implement fixed + per-seat + simple usage + hybrid; treat complex graduated pricing and real-time high-volume metering as later phases.

USP 4 — Unified business console and customer self-service

Business teams manage operations from a role-aware console. Customers see only their own plans, renewal dates, invoices, payments, usage and permitted plan changes. This is a shared data model with distinct interfaces, not two disconnected products.

USP 5 — Revenue and retention intelligence

Show MRR, ARR, active subscriptions, renewal rate, churn, overdue balances and plan performance. Add a renewal-risk indicator with transparent rule-based reasons in the MVP. Add trained churn prediction only when sufficient labelled historical data exists.

USP 6 — Reliable automation

Schedule renewals, generate invoices once, retry transient failures, prevent duplicate jobs, send notifications and preserve an audit trail. The system should make failures visible and recoverable.

USP 7 — Explainable finance assistant (advanced)

Allow natural-language questions over authorized business data, such as 'Which invoices are overdue?' or 'What is expected recurring revenue next month?' Responses should show the underlying filters and records. Do not allow the assistant to invent financial figures or execute sensitive actions without confirmation.

8. Functional Requirements and Website Structure

8.1 Public marketing website

Page	Content / actions
Home	Product proposition, core benefits, workflow preview, CTA to register/login
Features	Lifecycle, billing, reconciliation, analytics, automation and customer portal
Pricing	Optional platform pricing tiers if this product is sold as SaaS; distinguish platform fees from customers' own subscription plans
How it works	Product-to-payment lifecycle and role-specific explanation
Contact / About	Organization details, contact form and support information
Login / Register	Secure account access and registration

8.2 Admin and staff application — sidebar

Section	Screens and key capabilities
Overview	KPI cards; MRR/ARR trend; active subscriptions; renewals due; overdue invoices; recent activity; alerts
Products & Variants	Product list; create/edit product; type; sales price; cost price; notes; variants/attributes; price adjustment; archive
Plans & Pricing	Billing interval; amount; minimum quantity; availability dates; auto-close; pause/renew rules; trial; add-ons; usage/seat/credit configuration
Customers	Customer list; individual/company profile; contacts; billing address; tax details; status; subscriptions; invoices; payment history; notes
Quotations	Templates; line items; validity period; taxes; discounts; preview/PDF; send; accept/reject/expire; convert to subscription
Subscriptions	List and filters; subscription detail; status timeline; start/end dates; plan and line items; pause/resume; renew; upgrade/downgrade; cancel/close; activity log
Invoices	Draft/confirmed/paid/overdue/void states; invoice detail; line items; tax; discounts; PDF; send; confirm; cancel/void with reason; payment allocations
Payments	Record payment; method; amount; date; reference; notes; allocate to invoice; partial/multiple payments; refund record; unmatched payments
Reconciliation	CSV import; column mapping; match by reference/amount/customer/date; confidence indicator; manual review; matched/unmatched/duplicate states
Renewals & Automation	Calendar/list; due subscriptions; renewal rules; scheduled invoice jobs; retry queue; notification history; failed jobs
Discounts & Coupons	Fixed/percentage; eligible product/plan; minimum quantity/purchase; validity; usage cap; per-customer limit; automatic application
Analytics & Intelligence	MRR/ARR; revenue collected; outstanding; churn; renewal rate; ARPU; LTV estimate; plan/customer breakdown; forecast and scenario simulator
Notifications	In-app/email templates; renewal reminders; invoice notices; payment success/failure; overdue alerts; delivery status
Team & Roles	Invite staff; admin/sales/operations/finance roles; permission matrix; account status; staff audit
Settings	Business profile; currency; timezone; tax rules; invoice numbering; payment methods; email configuration; security; integrations; audit logs

8.3 Customer portal

Page	Content / actions
Customer Home	Active plans, next renewal, amount due, recent invoices, alerts
My Subscriptions	Active/paused/expired subscriptions; plan details; renewal date; permitted pause/cancel/change
Plans & Upgrades	Available plans; feature/price comparison; upgrade/downgrade request; proration preview if implemented
Usage & Credits	Usage to date, included quota, remaining credits and overage estimate (where supported)
Billing History	Invoice list, invoice detail, PDF download, payment status
Payments	Payment methods (if integrated), payment history, outstanding balance and receipts
Profile & Billing Details	Contact details, billing address, organization/tax fields
Notifications & Support	Renewal/payment notices, support request and billing query history

8.4 Role-based access control

Admin: full configuration and team management. Sales: customers, quotations and conversion; no tax/security configuration unless granted. Operations: subscription lifecycle and renewal exceptions. Finance: invoices, payments, reconciliation and financial reports. Customer: own account only. Enforce authorization in backend APIs, not only by hiding frontend menu items.

9. End-to-End Workflows and Business Rules

9.1 Customer registration and authentication

Enforce unique email, password policy from the challenge (more than eight characters, uppercase, lowercase and special character), secure password hashing, email-based reset token with expiry, access/refresh token handling, session revocation and role checks. Staff accounts can only be created by an authorized admin.

9.2 Product, variant and plan setup

Product stores name, type, sales price, cost price, notes and status. Variants can add a defined price adjustment (for example, selecting Odoo brand adds ■560). A plan stores billing interval (daily/weekly/monthly/yearly), base price, minimum quantity, availability window, trial/renewal/expiry behaviour and allowed lifecycle actions. Store plan versions so old subscriptions retain their agreed price.

9.3 Quotation-to-subscription

Sales selects customer and quotation template, adds products/plan lines, quantities, taxes and discounts, sets validity, previews and sends. Accepted quote converts into a subscription with a traceable source quotation. Expired/rejected quotes cannot be activated without an authorized revision.

9.4 Subscription state model

Challenge states: Draft → Quotation → Confirmed → Active → Closed. Additional operational states may include Paused, Past Due and Pending Cancellation. Define allowed transitions explicitly; record who changed state and why. Closed should be terminal unless a new subscription is created or an authorized reopen policy exists.

9.5 Billing and invoice generation

A billing run finds subscriptions due in the selected period, computes line items, quantity, usage, discounts and tax, creates one invoice per defined billing event, then records the billing period and idempotency key. Use a transaction so invoice creation and the subscription's next billing date cannot diverge. Re-running the same job must not create a duplicate invoice.

9.6 Taxes and discounts

Tax rules should be configurable by jurisdiction and product/customer tax category. For an India demo, GST should be represented as configurable tax components and rates; obtain qualified review before using for statutory invoicing. Discounts support fixed or percentage value, eligible products/subscriptions, minimum purchase/quantity, start/end date and usage caps. Apply eligible discounts automatically and record the calculation on the invoice.

9.7 Payment and reconciliation

A payment record contains method, amount, date, reference, note and status. Allocation links a payment to one or more invoices. Invoice balance = invoice total – allocated successful payments – approved credits/adjustments. Keep unmatched receipts separate until reviewed. Payment gateway webhooks must be signature-verified in production.

9.8 Renewal and failed-payment recovery

A scheduled worker identifies due subscriptions, applies renewal policy, generates billing once, sends reminders and records outcomes. Failed payment triggers a retry policy with bounded attempts and backoff; customer and staff receive notices. Subscription suspension/cancellation after repeated failures must follow the configured grace period and be auditable.

9.9 Cancellation, pause and plan change

Respect plan-level rules. Show effective date, refund/credit policy and any proration preview before confirmation. For MVP, implement immediate or end-of-period change as a clearly selected policy. Do not silently alter historical invoices or agreed subscription prices.

9.10 Audit trail

Log actor, action, entity, timestamp, previous value, new value and reason for sensitive changes such as price edits, discounts, invoice voids, refunds, role changes and subscription cancellation.

10. Analytics, Metrics and Intelligence

Metric	Definition / recommended calculation	Caution
MRR	Normalized monthly recurring value of active recurring subscriptions	Exclude one-time charges; define treatment of annual plans and discounts
ARR	MRR × 12 as a run-rate estimate	Not the same as cash collected or recognized accounting revenue
Collected revenue	Successful payments received in selected period	Use payment date and exclude failed/reversed payments
Outstanding receivables	Confirmed invoice balance still unpaid	Avoid double-counting partially paid invoices
Churn rate	Subscriptions lost during period ÷ opening active subscriptions	Document logo vs revenue churn and cohort rules
Renewal rate	Eligible subscriptions renewed ÷ subscriptions due for renewal	Define grace periods and renewal window
ARPU	Recurring revenue ÷ active customers for the period	Define customer vs account denominator
LTV estimate	ARPU × gross margin ÷ churn rate (simplified model)	Only an estimate; depends heavily on assumptions
Payment success rate	Successful payment attempts ÷ total payment attempts	Separate first-attempt success from recovered payments

10.1 Forecasting

A transparent baseline forecast can combine scheduled renewals, current recurring contract value, historical collection rates and known cancellations. Show forecast range and assumptions. A machine-learning forecast is a later enhancement requiring sufficient clean historical data.

10.2 Renewal-risk score

MVP rule-based signals can include repeated payment failures, overdue invoices, approaching expiry without renewal intent, repeated support disputes or low usage (if usage data exists). Show reasons and suggested next action. Do not describe a rule score as a trained predictive model.

10.3 What-if simulator

Allow finance/admin users to adjust hypothetical price, discount, renewal and churn assumptions and view the resulting projected MRR/ARR. Clearly label outputs as scenarios, not actual booked revenue.

11. Technical Architecture

11.1 Recommended hackathon stack

Layer	Recommended choice	Reason
Frontend	React + Tailwind CSS + Shadcn UI	Fast component-driven dashboard development
Backend	FastAPI (Python)	Fits rapid API development and Python analytics
Database	PostgreSQL	Relational integrity for customers, plans, invoices and allocations
Authentication	JWT access/refresh + password hashing	Role-aware API access
Background processing	Celery + Redis, or a simpler scheduled worker for MVP	Renewals, invoice runs, notifications and retries
Charts	Recharts	Dashboard and trend visualization
PDF	ReportLab or HTML-to-PDF library	Invoice and quotation documents
Deployment	Vercel frontend + Render/compatible backend and managed PostgreSQL	Practical demo deployment

11.2 Logical architecture

Browser (Admin / Staff / Customer) → React frontend → FastAPI REST API → domain services → PostgreSQL. A background worker consumes scheduled billing, renewal and notification jobs through Redis. Email and payment providers are adapters. Analytics queries read from normalized business records and pre-aggregated summaries where needed.

11.3 Core data entities

Entity	Key fields / relationships
User	id, email, password_hash, role, status
Customer	id, account_type, name, email, billing_address, tax_profile
Product	id, name, type, sales_price, cost_price, notes
ProductVariant	id, product_id, attributes, price_adjustment
Plan / PlanVersion	id, product_id, interval, price, min_quantity, availability, lifecycle_rules, version
Quotation / QuotationLine	id, customer_id, valid_until, status; product/plan, qty, price, tax, discount
Subscription / SubscriptionLine	id, customer_id, plan_version_id, state, start/end/next_bill dates; line quantities/prices
Invoice / InvoiceLine	id, subscription_id, billing_period, totals, tax, discount, status; immutable line snapshots
Payment	id, method, amount, date, reference, status
PaymentAllocation	payment_id, invoice_id, allocated_amount
DiscountRule / DiscountRedemption	type, value, eligibility, validity, usage caps; applied record
UsageEvent / CreditLedger	customer/subscription, event, quantity or credit movement, timestamp
Notification / Job	event, recipient, status, retry_count, scheduled_at
AuditLog	actor, action, entity, before/after, timestamp, reason

11.4 Reliability and correctness controls

Use database transactions for invoice creation and payment allocation; unique idempotency keys for billing events; immutable invoice snapshots; foreign keys and constraints; role checks at API level; bounded retries with exponential backoff; dead-letter/manual review for repeated job failures; timezone-aware billing dates; structured logs; pagination and indexes. Never store raw card data; use a compliant payment provider.

11.5 Performance strategy

Index customer_id, subscription status, next_billing_at, invoice status/due_date, payment reference and common filter columns. Paginate large tables, avoid N+1 queries, pre-aggregate dashboard metrics, cache safe read-heavy summaries and measure p50/p95 response time under seeded load. The brief's under-two-second target should be verified on the actual deployed environment.

12. Security, Privacy and Compliance Considerations

Enforce least privilege and tenant/customer data isolation. Hash passwords using a modern password-hashing algorithm; protect reset tokens; validate inputs; apply rate limits to login and sensitive endpoints; use HTTPS; store secrets in environment/secret management; verify payment webhooks; log security-relevant actions; back up data and test restoration.

Financial records should be auditable. Refunds, invoice voids and role changes should require explicit permissions and reasons. Do not store card PAN/CVV. For production use, obtain review for applicable GST invoicing, privacy/data-protection, retention, consumer cancellation and payment-provider obligations. This proposal is not legal or tax advice.

13. Feasibility and Implementation Roadmap

13.1 Hackathon MVP (build first)

Priority	Deliverable	Acceptance check
P0	Login, roles and seeded demo accounts	Admin, staff and customer see only permitted screens/data
P0	Products, variants and plans	Create/edit product and monthly/yearly plan; variant adjustment works
P0	Customer and quotation	Create customer, prepare quote, accept and convert
P0	Subscription lifecycle	Create subscription and move through valid states
P0	Invoice generation	Generate invoice from subscription with correct lines, tax and discount
P0	Payment recording	Record full/partial payment and calculate remaining balance correctly
P0	Overview dashboard	Show active subscriptions, billed/collected/outstanding and renewals due
P1	Renewal worker and reminders	Due subscription produces one invoice and one auditable job outcome
P1	CSV reconciliation	Import statement and match by reference/amount; unmatched items visible
P1	Customer portal	Customer views own subscriptions and invoices
P2	Risk scoring / forecast / assistant	Show transparent rules and assumptions; label demo data

13.2 Suggested build sequence

1) Define schema, roles and sample data. 2) Build authentication and navigation. 3) Implement product/plan/customer CRUD. 4) Implement quotation and conversion. 5) Implement subscription lifecycle and billing calculation. 6) Implement invoice and payment allocation. 7) Add dashboard and customer portal. 8) Add renewal job, CSV reconciliation and notifications. 9) Seed demo scenarios, test edge cases and rehearse the presentation.

13.3 Feasibility assessment

Area	Hackathon feasibility	Approach
Core CRUD and lifecycle	High	Implement with relational tables and explicit state transitions
Invoice/tax/discount calculations	High for demo	Use deterministic rules and test cases; label jurisdiction assumptions
Payments	High for simulated/manual records; medium for live gateway	Use sandbox or mock adapter; do not claim real settlement if simulated
Renewal automation	Medium-high	Scheduled worker with idempotency and job history
CSV reconciliation	Medium-high	Implement a controlled import and review workflow
Usage/credit billing	Medium	Start with event ingestion and simple metering; advanced scale later
Predictive churn ML	Low without historical data	Use explainable rules for MVP; ML roadmap later
Production-grade compliance/scale	Low within one hackathon	Document as future engineering and validation work

13.4 Risks and mitigations

Risk	Mitigation
Scope too large	Deliver a complete happy path before advanced features
Duplicate invoices	Unique billing-event key and transactional processing
Incorrect financial totals	Centralized calculation service and automated tests
Unauthorized data access	Backend role checks and customer ownership filters
Fake AI claims	Label rules-based outputs; use ML only with trained/validated data
Unreliable demo integrations	Provide sandbox/mock adapters and clearly label simulation
Slow dashboard	Pagination, indexes, aggregates and load testing

14. Demonstration Script for Judges

Use one realistic customer journey to prove that all modules work together. Avoid navigating through disconnected screens without showing data flow.

Step	Demo action	What the judge should see
1	Admin creates a product with a variant and a monthly plan	Product configuration and price adjustment
2	Sales creates a customer and quotation from a template	Quote validity, line items, tax and discount
3	Customer accepts quotation	Quote converts to a subscription with linked records
4	System activates subscription and generates invoice	Correct billing period, totals and invoice number
5	Finance records a partial payment	Invoice shows partially paid and remaining balance
6	Finance imports a CSV payment statement	Matched and unmatched transactions are visible
7	Advance demo clock / trigger due job	Renewal job runs once and creates the next invoice
8	Customer logs into portal	Only own subscription, invoice and payment are visible
9	Admin opens analytics	MRR, active subscriptions, collected revenue, outstanding and renewals due
10	Show exception case	Failed payment, retry queue, alert and audit trail

Prepare one successful path and at least two exception paths: partial payment and duplicate billing attempt. These demonstrate business correctness more convincingly than a large number of static pages.

15. Success Metrics and Validation

Area	Validation metric
Usability	A staff user can create a quote and convert it without spreadsheet/manual re-entry
Billing correctness	Invoice line, tax, discount and total match expected test fixtures
Duplicate prevention	Replaying the same billing job creates no duplicate invoice
Payment correctness	Partial and multiple allocations produce the correct outstanding balance
Security	Customer A cannot access Customer B's subscription/invoice through UI or API
Automation	Due renewal is processed once; failures are visible and retryable
Performance	Measured p95 page/API response under 2 seconds on a documented test dataset/environment
Analytics	Dashboard metrics reconcile to source invoice/payment/subscription records

Do not present unmeasured targets as achieved results. Record test dataset size, environment, test duration and observed latency when reporting performance.

16. Future Scope

After the MVP, expand toward: production payment gateway integrations and verified webhooks; accounting/CRM integrations; configurable multi-entity and multi-brand support; advanced tax jurisdictions; richer usage metering and credit ledger; proration and complex tier pricing; dunning campaigns; customer segmentation; trained churn and revenue forecasting models; anomaly detection; natural-language analytics with permission-aware query generation; mobile-friendly portal; localization; and enterprise audit/compliance controls.

17. Conclusion

The proposed platform addresses the Brightloop challenge by replacing disconnected spreadsheets and manual processes with a connected subscription operating workflow. The reference review shows complementary strengths across usage monetization (FlexPrice), recurring expense visibility (Huhusmang and Subflow), business billing operations (BillaBear) and backend reliability (AnuragThePathak).

The project's strongest defensible differentiation is not that it claims to contain every feature missing from competitors. It is the combination of business lifecycle management, transparent payment reconciliation, role-specific customer and staff experiences, flexible pricing and actionable revenue intelligence in one coherent product. The hackathon implementation should prioritize correctness of the complete product-to-payment journey, then add automation and intelligence.

18. References

1. FlexPrice, GitHub repository and README: https://github.com/flexprice/flexprice

2. Huhusmang, Subscription Management, GitHub repository and README: https://github.com/huhusmang/Subscription-Management

3. BillaBear, GitHub repository and README: https://github.com/billabear/billabear

4. Subflow, GitHub repository and README: https://github.com/ridemountainpig/subflow

5. AnuragThePathak, Subscription Management, GitHub repository and README: https://github.com/AnuragThePathak/subscription-management

6. Supplied hackathon problem statement: Subscription_Management_System.pdf (Brightloop Software scenario and functional requirements).

Research note: Public README and repository documentation were reviewed on 31 October 2026. Repository contents and features can change; re-check current code and documentation before making implementation-level or competitor-exclusion claims.

Appendix A. Two-Second Performance Requirement

Purpose. The Brightloop challenge states that pages should respond in under two seconds even when the system contains thousands of subscriptions. This is a user-facing responsiveness requirement: staff and customers should be able to open important screens, search records, apply filters and complete common interactions without waiting several seconds for usable results.

A.1 What “under 2 seconds” actually means

It does not mean every background task—such as sending email, generating a large PDF, importing a bank statement or processing a scheduled renewal—must finish in two seconds. It means the application should acknowledge the user action and render the requested page or usable result within the target. Long-running work should be queued and shown with a clear progress or status indicator.

Measure both (1) the time until the main page content becomes usable in the browser and (2) the API response time that supplies that content. Do not measure only the server while ignoring a slow frontend, or only a cached page while ignoring real database queries.

A.2 Screens and operations that must be tested

Screen / operation	What should respond	Acceptance target
Finance dashboard	Summary cards and key charts for revenue, active subscriptions, overdue invoices and renewals	Usable initial view < 2 seconds
Subscription listing	Paginated list with status, plan, customer and next billing date	Initial results and common filters < 2 seconds
Customer search	Search by customer name, email or supported identifier	Search results < 2 seconds
Invoice listing	Paginated invoices with status, due date, customer and amount	Initial results and common filters < 2 seconds
Product and plan listing	Configured products, variants and plans	Initial results < 2 seconds
Subscription detail	Customer, plan, status, billing history and recent activity	Usable detail view < 2 seconds
Common actions	Open details, change filter, next page, save normal form	Visible response/acknowledgement < 2 seconds where synchronous

Apply the target to all important user-facing pages in the challenge scope, not just the dashboard. The list above is the minimum demonstration checklist; test every additional major MVP page as well.

A.3 How the system will achieve the target

- **Database indexing:** Create indexes on fields used for joins, sorting and filters: customer_id, subscription status, next_billing_at, plan_id, invoice status, due_date and payment reference. Add composite indexes only for frequent query patterns confirmed by the application.
- **Pagination:** Never load thousands of subscriptions or invoices in one response. Use server-side pagination with a sensible page size (for example, 20–50 records) and return only the fields required by the screen.
- **Efficient queries:** Avoid N+1 queries. Fetch related customer and plan data efficiently, select only needed columns, inspect slow queries and use database query plans to tune the highest-cost endpoints.
- **Dashboard summaries:** Do not calculate every metric by scanning all subscriptions on every page load. Maintain summary/aggregate tables or cached metrics and refresh them when relevant billing events occur or through a scheduled job.
- **Caching:** Cache safe, frequently requested summary data where appropriate. Invalidate or refresh it when payments, invoices, renewals or subscription states change. Do not serve stale balances as authoritative financial records.
- **Frontend performance:** Load the main content first, avoid rendering huge tables at once, use table pagination, defer non-critical charts and split heavy screens where appropriate. Show skeleton/loading states without blocking the whole application.
- **Background processing:** Run renewals, invoice batches, email notifications, imports and large exports in background workers. Return a job/status identifier quickly and let the user view progress or completion.
- **Monitoring:** Record endpoint latency, database query time and browser page-load timing. Track p50 and p95 latency, errors and slow endpoints so optimization is based on evidence.

A.4 Performance test plan for the hackathon

1. Prepare realistic seed data with at least 5,000 subscriptions. If time and the database environment permit, use 10,000 subscriptions as the stronger demonstration dataset. Include multiple customers, plans, statuses, billing dates and invoices so filters and joins are exercised.
2. Run the test against the actual application stack and database configuration used for the demo. Record the hardware/runtime environment, database size, record counts, browser, network conditions and whether caches are warm or cold.
3. Test each major page and operation in A.2 repeatedly, not just once. Include dashboard load, subscription list, status/date filters, customer search, invoice list and subscription detail.
4. Record median (p50) and p95 response times. The recommended acceptance criterion is p95 below 2.0 seconds for the tested core page/API operations under the stated demo conditions. Also report the slowest observed request and error rate.
5. Repeat the test after optimization and retain the results (screenshots, browser timings, API logs or a small benchmark report) as evidence for judges.

A.5 Recommended measurable acceptance criteria

Metric	Recommended target	How to verify
Core user-facing page load	Under 2.0 seconds	Browser timing: navigation to usable main content
Core API response	p95 under 2.0 seconds	Server/API logs or API load-test report
Subscription/invoice list	Under 2.0 seconds for first page and common filters	Seeded thousands of records
Customer search	Under 2.0 seconds	Search representative names/emails in seeded data
Data volume	At least 5,000; aim for 10,000 subscriptions	Database seed count and benchmark report
Correctness under load	No duplicate invoices or incorrect allocations	Repeat billing/payment actions; verify constraints and audit records

Important distinction. The original challenge gives the requirement as “under two seconds” with “thousands of subscriptions”; it does not specify a precise record count, p95 percentile, concurrency level or test hardware. The 5,000/10,000 data-volume suggestions and p95 criterion above are recommended ways to make the requirement measurable for the hackathon. They are proposed validation criteria, not claims that the current prototype has already achieved them.

A.6 What to show the judges

- Show the seeded record count (for example, 10,000 subscriptions) in the database or a controlled admin view.
- Open the finance dashboard, subscription list, invoice list and customer search while recording page/API timings.
- Apply a status/date filter and open a subscription detail page to show that performance remains responsive beyond the dashboard.
- Show benchmark results with p50, p95, slowest request, error rate and test conditions. Do not claim “under 2 seconds” unless recorded results support it.
- For asynchronous tasks, demonstrate quick acknowledgement, queued/processing/completed status and duplicate prevention when a task is retried.

Suggested judge-facing explanation: “We interpreted the two-second requirement as the time users wait for core pages and common operations to become usable, even with thousands of subscription records. We use indexed queries, server-side pagination, optimized dashboard summaries and background workers for heavy jobs. We validate it using a seeded dataset and report measured p95 timings instead of making an unverified performance claim.”